

APOSTILA DO ALUNO — AULA 8

UML — Diagrama de Sequência

Curso: Técnico em Desenvolvimento de Sistemas | Turma: 103513 | Disciplina: Modelagem de Sistemas (82h + 04h planejamento)

Data: 09/10/2026 (sexta-feira) | Horário: 07:30 às 11:30 (4h)

Objetivos desta aula

- Rever, de forma breve, o diagrama de atividades da Aula 7 e situar a aula de hoje entre os diagramas comportamentais da UML.
- Explicar o que é um Diagrama de Sequência, para que serve e como ele se diferencia do diagrama de atividades e do diagrama de classes.
- Reconhecer e usar os elementos: participante, ator, linha de vida, ativação, mensagens síncrona, assíncrona e de retorno, auto-mensagem, criação e destruição de objetos.
- Usar os fragmentos combinados alt, opt e loop, com condições de guarda.
- Relacionar o diagrama de sequência com as regras de negócio (Aula 1), os casos de uso (Aula 4), as classes (Aulas 5 e 6) e as atividades (Aula 7).
- Modelar, em grupo, o cenário “pedido pelo app com PIX” do TechBurger (Atividade 1).
- Encontrar e corrigir erros em um diagrama de sequência.
- Revisar, em duplas, tudo o que foi visto nas Aulas 1 a 8 aplicando ao cenário CineVila (Atividade 2).

Revisão breve: Aula 7 — Diagrama de Atividades

Na Aula 7 desenhamos o FLUXO de um processo: ações (verbo + complemento) ligadas por setas, do nó inicial até um nó final; decisões com condições de guarda completas e exclusivas; junções que reúnem caminhos alternativos; bifurcação e união para passos em paralelo; e raias mostrando QUEM executa cada ação. As regras de negócio da Aula 1 viraram decisões dentro do processo — por exemplo, [valor total ≥ R\$ 15,00?].

Elemento (Aula 7)	Para que serve	Como vai reaparecer hoje
Decisão com guardas	Escolher UM caminho	Fragmento alt, com as mesmas guardas
Decisão que “pula” um trecho	Fazer algo só às vezes	Fragmento opt
Laço (seta que volta)	Repetir passos	Fragmento loop
Bifurcação / união	Passos em paralelo	Fragmento par (ou mensagem assíncrona)
Raias	Quem executa cada ação	Linhas de vida: quem participa da conversa

O diagrama de atividades responde “o que acontece e em que ordem?”. Hoje vamos olhar o MESMO processo por outro ângulo: quem conversa com quem, dentro do sistema, e em que ordem essas conversas acontecem.

Quiz relâmpago — revisão da Aula 7

Testem o que lembram. Respostas no Gabarito.

1. Qual a diferença entre decisão e bifurcação?

2. O que é uma condição de guarda? Por que ela precisa ser completa?
3. Qual a diferença entre nó final da atividade e nó final de fluxo?
4. Como se representa um laço no diagrama de atividades?
5. Em que raia vai a ação “Aplica 5% de desconto”? Por quê?
6. Qual elemento espera TODOS os caminhos paralelos terminarem?

1. O Diagrama de Sequência

1.1 O que é

O Diagrama de Sequência é um diagrama COMPORTAMENTAL da UML que mostra a TROCA DE MENSAGENS entre os participantes de um cenário. Os participantes — atores e objetos — ficam lado a lado no topo do diagrama, e de cada um desce uma linha tracejada, a linha de vida. As mensagens são setas horizontais entre essas linhas, e o tempo corre de CIMA para BAIXO: uma mensagem desenhada mais abaixo acontece depois de uma desenhada mais acima. Por isso, no diagrama de sequência, a posição vertical faz parte do significado.

Normalmente cada diagrama de sequência mostra UM cenário de um caso de uso — por exemplo, “pedido pago com PIX e aprovado”. Os desvios mais importantes podem entrar com fragmentos (seção 3), mas o objetivo não é desenhar todos os caminhos possíveis: isso é papel do diagrama de atividades.

1.2 Para que serve

- Detalhar COMO os objetos colaboram para realizar um caso de uso da Aula 4.
- Descobrir os métodos de que cada classe precisa e conferir se eles existem no diagrama de classes.
- Deixar clara a ORDEM das chamadas: validar antes de cobrar, cobrar antes de mandar para a cozinha.
- Documentar integrações entre sistemas: o app conversando com o sistema de pagamento, com a cozinha, com o servidor.
- Guiar quem programa e quem testa: cada mensagem é uma chamada de método a implementar e a verificar.

No mercado, o diagrama de sequência é muito usado para documentar APIs e integrações (aplicativo ↔ servidor ↔ gateway de pagamento), fluxos de login e autorização (como o “Entrar com Google”), a comunicação entre microsserviços e o combinado entre as equipes de front-end e back-end antes de programar. Também é uma ferramenta de investigação: quando um erro aparece “só às vezes”, desenhar a sequência de chamadas costuma revelar uma resposta que chegou antes da hora ou uma chamada que ninguém esperava.

1.3 Atividades x sequência

	Diagrama de Atividades (Aula 7)	Diagrama de Sequência (hoje)
Foco	O FLUXO do processo	A CONVERSA entre os participantes
Organização	Raias = quem executa	Linhas de vida = quem participa
Bom para	Regras e decisões do negócio	Ordem das chamadas e integrações
Quem lê	Pessoas do negócio e da TI	Principalmente a equipe técnica
Ligação com o código	Indireta	Direta: cada mensagem é um método

1.4 Classes x sequência

O diagrama de classes das Aulas 5 e 6 é uma fotografia do sistema parado: mostra as classes, seus atributos, seus métodos e como elas se ligam. O diagrama de sequência é o filme do sistema em ação: mostra esses métodos SENDO CHAMADOS, um depois do outro. Por isso os dois precisam concordar: se o :App envia a mensagem aplicarDesconto() para o :Pagamento, a classe Pagamento precisa ter o método aplicarDesconto() — e ela tem, desde a Aula 5.

2. Elementos do Diagrama de Sequência

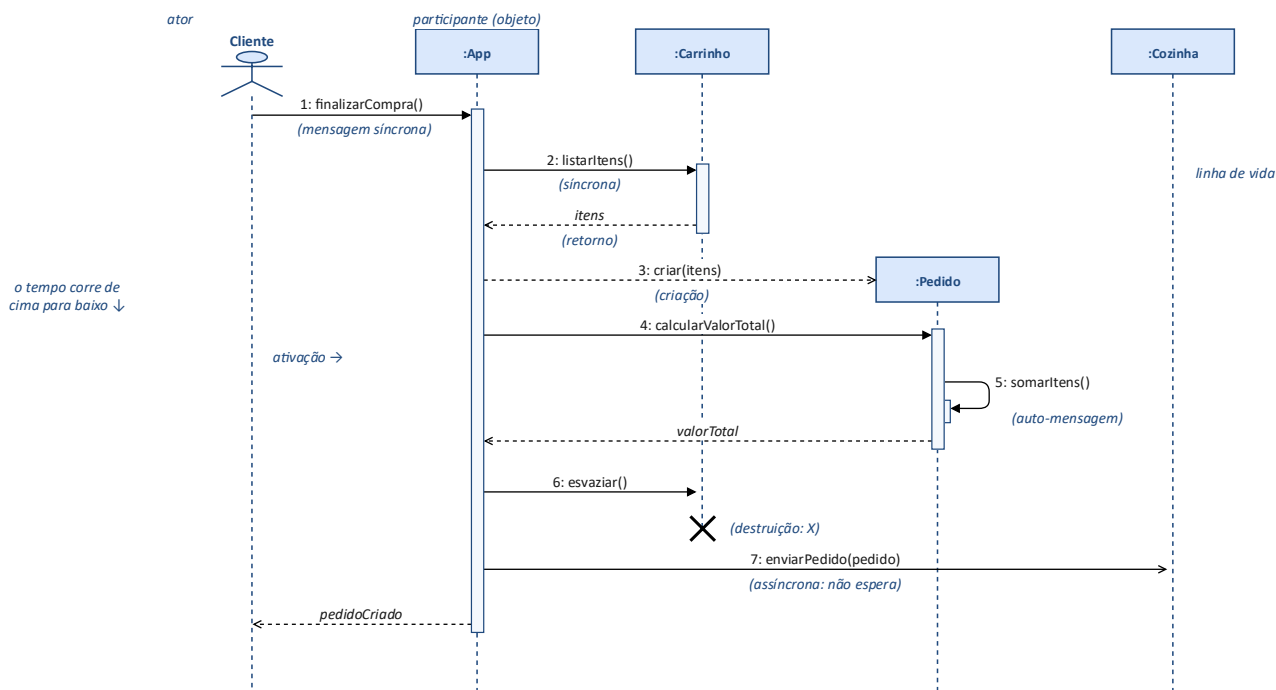


Figura 1 — Anatomia de um diagrama de sequência.

Antes de ver cada peça em detalhe, vale olhar o conjunto. As peças do diagrama de sequência se dividem em dois grupos. O primeiro grupo responde QUEM conversa: o ator (o bonequinho, a pessoa de fora), os participantes (as caixas “:Classe”, objetos do sistema), a linha de vida de cada um (o tracejado que desce) e a ativação (a barrinha que mostra que o participante está trabalhando). O segundo grupo responde COMO eles conversam: as mensagens — síncrona (chama e espera), de retorno (a resposta que volta), assíncrona (manda e não espera), a auto-mensagem (o objeto chama a si mesmo) e as setas de criação e destruição de objetos. Mais adiante (seção 3) entram os fragmentos combinados, as molduras que mostram o SE e a REPETIÇÃO. A tabela abaixo resume todas as peças; as seções 2.1 a 2.5 detalham uma por uma.

Elemento	Símbolo	Significado
Participante (objeto)	Caixa no topo com “:Classe”	Um objeto que participa da conversa. A classe existe no diagrama de classes.
Ator	Bonequinho no topo	Quem está fora do sistema e inicia a conversa (Aula 4).
Linha de vida	Linha tracejada vertical	A existência do participante ao longo do tempo.
Ativação	Retângulo fino sobre a linha de vida	O período em que o participante está trabalhando.
Mensagem síncrona	Seta contínua, ponta cheia	Chamada de método; quem envia espera a resposta.

Mensagem assíncrona	Seta contínua, ponta aberta	Quem envia não espera; manda e segue em frente.
Mensagem de retorno	Seta tracejada, ponta aberta	Devolve o resultado para quem chamou.
Auto-mensagem	Seta que sai e volta ao mesmo objeto	O objeto chama um método dele mesmo.
Criação	Seta tracejada até a caixa do objeto	O objeto nasce naquela altura do diagrama.
Destruição	X no fim da linha de vida	Dali para baixo o objeto não existe mais.

2.1 Participantes e atores

Cada participante é escrito como “:NomeDaClasse” — :Pedido, :Pagamento, :Aula. Os dois-pontos indicam que se trata de um OBJETO daquela classe (se for preciso distinguir dois objetos da mesma classe, dá para dar um nome: p1:Pedido). Sistemas externos também podem ser participantes, como o :SistemaPagamento. O ator, desenhado como um bonequinho, é quem está FORA do sistema e começa a conversa; ele vem do diagrama de casos de uso da Aula 4 e normalmente ocupa a primeira coluna, à esquerda.

Atenção a uma confusão comum: o ator Cliente é a PESSOA que usa o app; o objeto :Cliente são os DADOS dessa pessoa dentro do sistema (nome, pontos de fidelidade...). Os dois podem aparecer no mesmo diagrama, em colunas diferentes.

Quem pode ser participante? Quatro tipos aparecem com frequência: (1) um ATOR, a pessoa ou papel de fora que começa a conversa — Cliente, Bibliotecário; (2) a “porta de entrada” do sistema, o app ou a tela que atende o ator — :App, :SistemaBiblioteca; (3) um OBJETO de uma classe do diagrama de classes das Aulas 5 e 6 — :Pedido, :Carrinho, :Livro; (4) um SISTEMA EXTERNO com quem o nosso conversa — :SistemaPagamento, o banco do PIX. O que NÃO é participante: uma ação (“Pagar”) ou um dado solto (valorTotal). Ações viram mensagens, e dados viram parâmetros ou retornos.

2.2 Linha de vida e ativação

A linha de vida é a linha tracejada que desce de cada participante. Toda mensagem sai de uma linha de vida e chega em outra (ou na mesma). Sobre a linha de vida desenhamos a ativação: um retângulo fino que começa quando o participante recebe uma mensagem e termina quando ele devolve o resultado. Enquanto a ativação do :App está aberta, ele está “ocupado” atendendo aquele pedido. Se um objeto chama um método dele mesmo, abre-se uma segunda barrinha por cima da primeira.

Como o tempo corre de cima para baixo, numerar as mensagens (1, 2, 3...) ajuda muito a leitura e a correção. Os retornos normalmente não recebem número: eles apenas fecham a chamada que está logo acima. A distância vertical entre as setas não representa tempo real — só a ORDEM importa.

2.3 Mensagens: síncrona, assíncrona e de retorno

A mensagem síncrona (seta contínua com ponta cheia) é uma chamada de método: quem envia ESPERA a resposta antes de continuar. É a mais comum. Exemplo: “:App → :Pedido: calcularValorTotal()”. A mensagem de retorno (seta tracejada com ponta aberta) devolve o resultado da chamada para quem chamou — “valorTotal”, “senhaOk”, “resultado” — e fica sempre ABAIXO da chamada que respondeu. Na UML o retorno é opcional; neste curso vamos desenhá-lo sempre que o resultado importar para o cenário.

A mensagem assíncrona (seta contínua com ponta ABERTA) é enviada sem espera: quem manda segue em frente. É o caso do :App enviando o pedido para a :Cozinha — o app responde ao cliente sem esperar o

hambúrguer ficar pronto — e das notificações por e-mail, SMS ou aviso no celular. Uma boa pergunta para decidir entre as duas: “quem envia precisa da resposta para continuar?”. Se sim, é síncrona.

2.4 Auto-mensagem, criação e destruição

A auto-mensagem é uma seta que sai e volta para a mesma linha de vida: o objeto chama um método dele mesmo, como o :Pedido que, ao calcular o valor total, chama o próprio método `somarItens()` (veja a Figura 1). Use quando o passo interno ajudar a entender o cenário. A criação é uma seta tracejada que chega na CAIXA do novo objeto: por isso a caixa dele aparece no meio do diagrama, e não no topo. Antes dessa seta o objeto não existe, e nenhuma mensagem pode chegar nele. O rótulo costuma ser «create» ou “criar(...)”. A destruição é um X no fim da linha de vida; dali para baixo o objeto não existe mais, e mandar uma mensagem para ele é erro de modelagem.

2.5 Como nomear as mensagens

- **Chamadas:** verbo no infinitivo + parênteses, com os parâmetros dentro — `validar()`, `cobrar(valor)`, `aplicarDesconto()`.
- **Mesmos nomes das classes:** use exatamente o nome do método que está no diagrama de classes da Aula 5, com a mesma grafia.
- **Retornos:** o que volta — `valorTotal`, `resultado`, `pedidoConfirmado`.
- **Nunca um substantivo solto numa chamada:** “pagamento” não diz o que fazer; “`cobrar(valorTotal)`” diz.
- **Toda seta tem nome:** uma seta sem rótulo não comunica nada.

2.6 Um exemplo simples

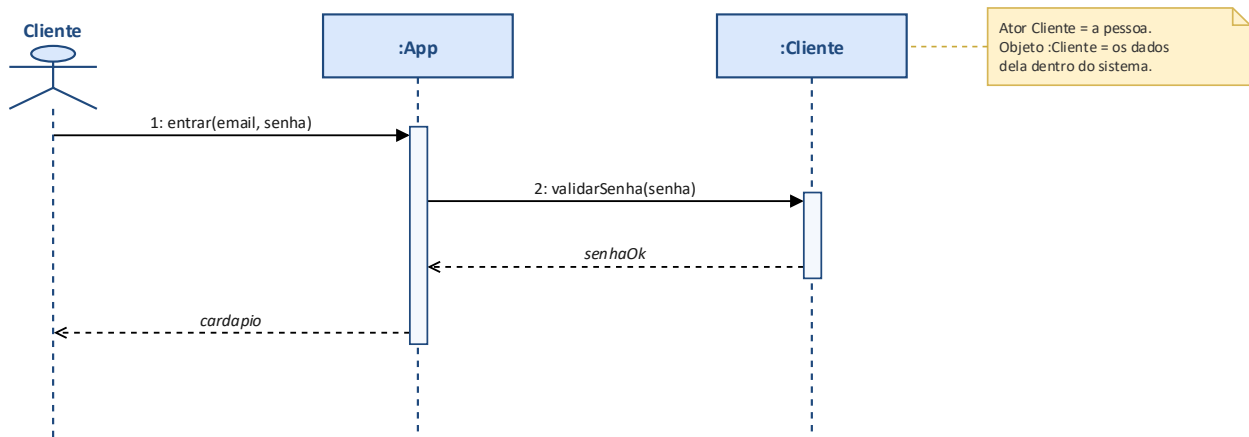


Figura 2 — Diagrama de sequência “login no app do TechBurger”.

Leitura: o ator Cliente chama o :App com “1: entrar(email, senha)” e a ativação do :App começa. O :App pede ao objeto :Cliente (os dados da pessoa) “2: validarSenha(senha)”;

o :Cliente devolve “senhaOk” pelo retorno tracejado, e a ativação dele termina. Por fim, o :App devolve o cardápio ao ator. É o mesmo login da Aula 7, agora como conversa entre objetos.

Quiz rápido: qual elemento?

Respostas no Gabarito.

1. A caixa no topo com “:Pedido” e a linha tracejada que desce dela?

2. O retângulo fino sobre a linha tracejada?
3. A seta contínua com ponta cheia?
4. A seta tracejada que volta com o resultado?
5. O X no fim de uma linha de vida?
6. A seta que sai e volta para o mesmo objeto?

3. Fragmentos combinados

E quando o cenário tem um SE, um SENÃO ou uma REPETIÇÃO? Usamos fragmentos combinados: uma moldura retangular em volta de um trecho das mensagens, com uma etiqueta no canto dizendo o tipo (alt, opt, loop, par, ref) e, junto dela, a condição de guarda entre colchetes — exatamente como as guardas da Aula 7.

3.1 alt — alternativas (se / senão)

A moldura alt é dividida por linhas tracejadas em dois ou mais trechos, cada um com a sua guarda. Só UM dos trechos acontece, como na decisão do diagrama de atividades — e as guardas seguem a mesma regra de ouro: completas e exclusivas. No último trecho pode-se usar [senão] para “todos os outros casos”.

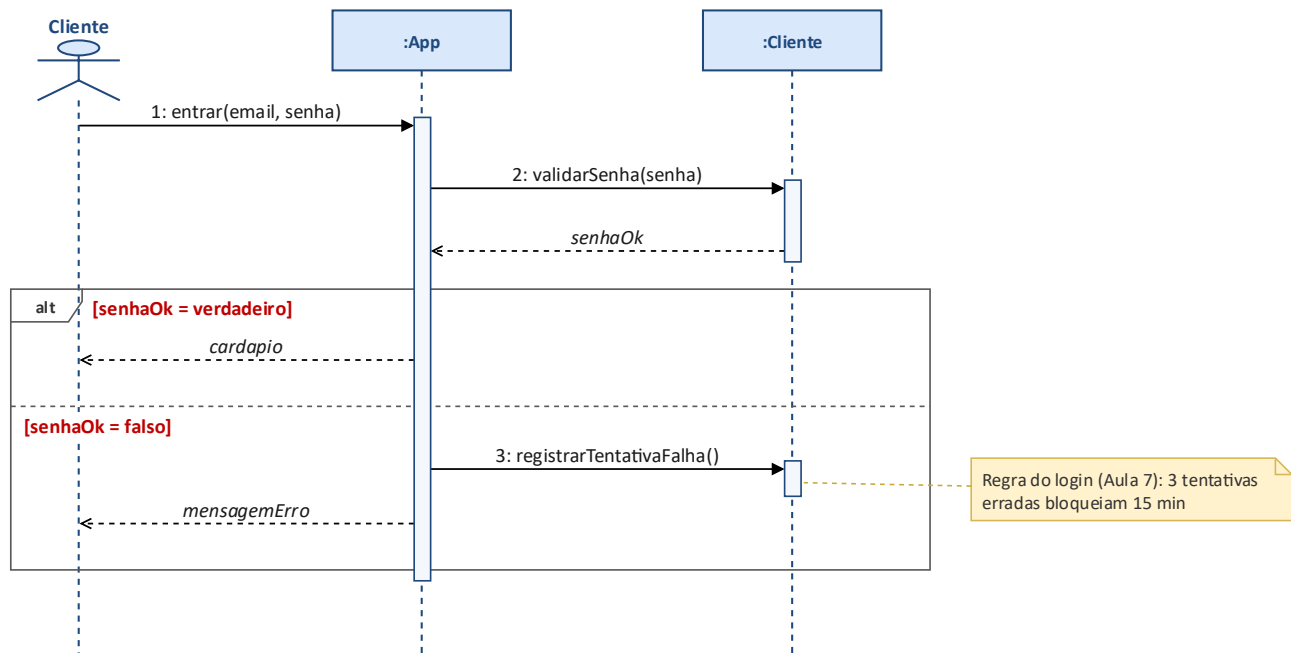


Figura 3 — alt no login: senha certa ou errada.

Leitura: as mensagens 1 e 2 acontecem sempre, antes da moldura. Com [senhaOk = verdadeiro], o :App devolve o cardápio; com [senhaOk = falso], registra a tentativa falha no :Cliente e devolve a mensagem de erro. A nota amarela lembra a regra do laço de login da Aula 7: três tentativas erradas bloqueiam o acesso por 15 minutos.

3.2 opt — opcional (só se)

A moldura opt tem um único trecho e uma guarda: ele só acontece SE a condição for verdadeira; se for falsa, o trecho é simplesmente pulado. É um alt sem o “senão”. Lembra do «extend» da Aula 4, que acontece “só às vezes”? Ele costuma virar um opt: Aplicar Desconto PIX estende Confirmar Pagamento, e aqui aparece como opt [forma = PIX].

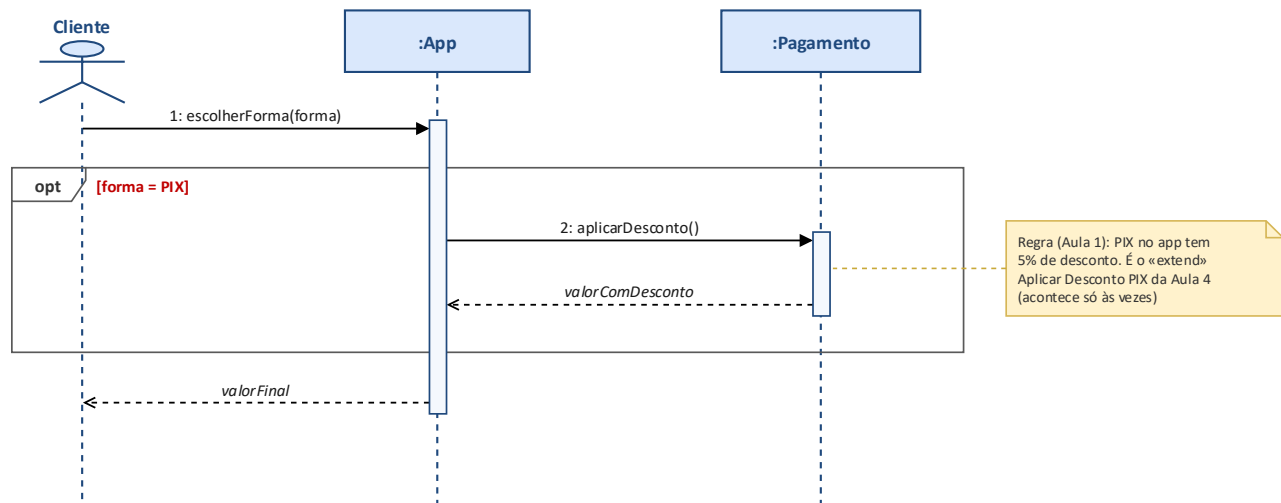


Figura 4 — opt: o desconto do PIX.

3.3 loop — repetição

A moldura loop repete o trecho de dentro enquanto a guarda for verdadeira. A guarda diz quantas vezes: [para cada item escolhido], [até 3 tentativas]. É o laço da Aula 7 — lá era uma seta que voltava para uma junção; aqui é uma moldura. Tudo o que vem depois da moldura acontece uma vez só, quando a repetição termina.

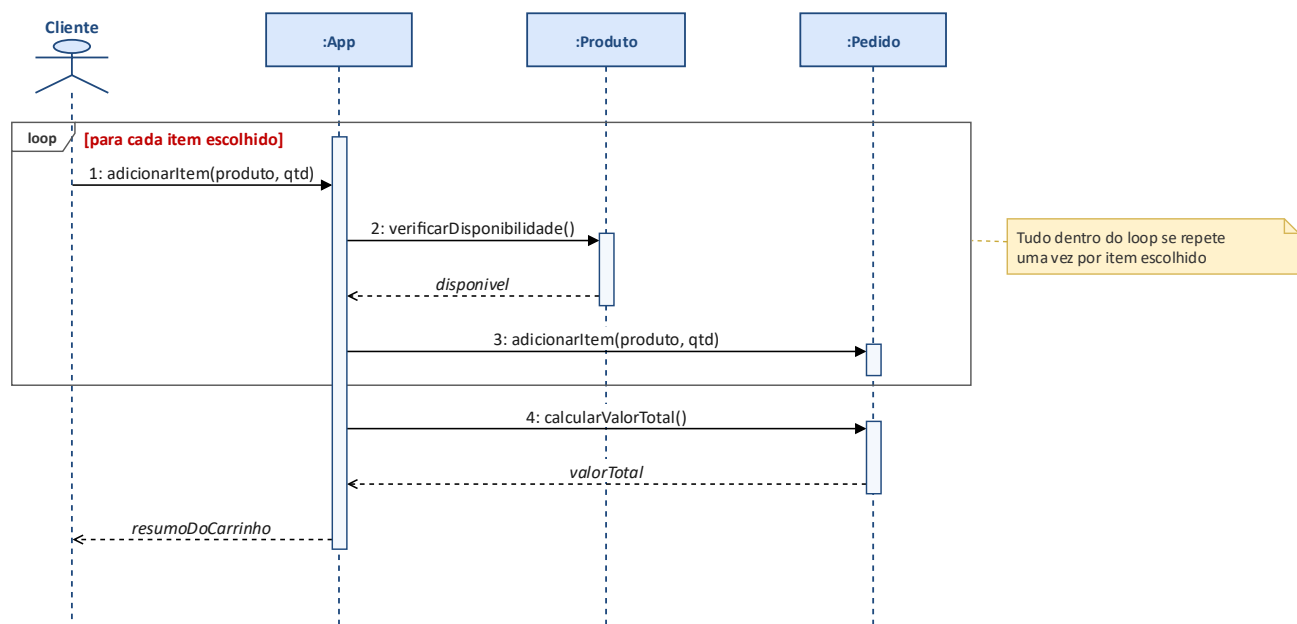


Figura 5 — loop: adicionando itens ao carrinho.

3.4 Outros fragmentos e a comparação com a Aula 7

Para reconhecer quando aparecerem: par (trechos em paralelo, o equivalente à bifurcação/união), ref (referência a outro diagrama, como “ref Fazer Login”, para não redesenhar o login toda vez) e break (interrompe o cenário quando a guarda é verdadeira). Fragmentos podem ficar um dentro do outro — um opt dentro de um alt, por exemplo.

No diagrama de atividades (Aula 7)	No diagrama de sequência (hoje)
Decisão com duas saídas	alt com dois trechos
Decisão com uma saída que “pula” um trecho	opt
Seta que volta para uma junção (laço)	loop
Bifurcação e união	par (ou mensagens assíncronas)

Quiz rápido: qual fragmento?

Para cada frase, digam qual fragmento usar. Respostas no Gabarito.

1. “Se o pagamento for aprovado, confirma o pedido; senão, cancela.”
2. “Para cada item do carrinho, verifica o estoque.”
3. “Se o cliente tiver cupom, aplica o desconto.” (Se não tiver, não há nada a fazer.)
4. “O sistema envia o e-mail e o SMS ao mesmo tempo.”
5. “O login completo está em outro diagrama; aqui só apontamos para ele.”

4. O diagrama de sequência e as aulas anteriores

4.1 Das classes às mensagens (Aulas 5 e 6)

Todo participante “:X” é um objeto de uma classe X do diagrama de classes, e toda mensagem RECEBIDA por um objeto deve ser um MÉTODO da classe dele. Se a mensagem não existe como método, falta alguma coisa no diagrama de classes — e é hora de corrigir. Os atributos derivados aparecem nas perguntas que os objetos respondem: /livrosEmprestados (Associado), /vagasDisponiveis (Aula), /pagamentosEmDia (Aluno).

Classe (Aula 5)	Métodos que usaremos hoje
Cliente	acumularPontos(valor), resgatarCombo()
Pedido	calcularValorTotal(), confirmar()
Pagamento	aplicarDesconto(), confirmar()
Produto	verificarDisponibilidade(), sugerirSubstituto()
Associado / Professor	podePegarLivro(), prazoEmprestimo() (Professor redefine: 30 dias)
Livro / Empréstimo	estaDisponivel() / calcularMulta()
Aluno / Aula	reservarVaga(), /pagamentosEmDia / temVagaDisponivel(), /vagasDisponiveis

4.2 Dos casos de uso aos cenários (Aula 4)

Cada caso de uso importante pode ganhar um ou mais diagramas de sequência — um para cada cenário relevante. O ator do caso de uso é o primeiro participante, à esquerda. Um «include» (sempre acontece) vira mensagens que estão sempre presentes; um «extend» (só às vezes) vira um fragmento opt.

4.3 Das regras de negócio às guardas (Aula 1)

Regra de negócio (Aula 1)	Como aparece no diagrama de sequência
3 cancelamentos no mês bloqueiam o app por 15 dias	alt [estaBloqueado() = verdadeiro] → o pedido é recusado
PIX pelo app tem 5% de desconto	opt [forma = PIX] com aplicarDesconto()
Só vai para a cozinha com pagamento confirmado	alt [resultado = aprovado] antes de enviarPedido()
Mais de 10 itens: confirmação do gerente	opt [mais de 10 itens] com aguardarGerente()
1 ponto a cada R\$ 10,00; 100 pontos = combo	acumularPontos() + opt [saldo ≥ 100 pontos]
No máximo 3 livros emprestados	podePegarLivro() + alt [pode e disponível]

4.4 Atividades e sequência: o mesmo processo

O diagrama de atividades da Aula 7 e o diagrama de sequência de hoje descrevem o MESMO pedido. Cada ação importante das raias vira uma ou mais mensagens entre objetos; as decisões viram alt ou opt; o laço vira loop; o paralelo vira par. O diagrama de sequência vai além, mostrando QUAL objeto faz cada coisa e em que ordem. Os dois precisam concordar: se um deles cobra o cliente antes de validar o pedido, os dois estão errados.

4.5 Passo a passo para construir um diagrama de sequência

1. Escolham UM cenário de um caso de uso e escrevam o nome dele (ex.: “pedido com PIX aprovado”).
2. Listem os participantes: o ator primeiro, depois o :App e os objetos das classes envolvidas.
3. Coloquem os participantes lado a lado, na ordem em que entram na conversa.
4. Desenhem as mensagens de cima para baixo, numeradas, com os nomes dos métodos.
5. Acrescentem os retornos tracejados e as ativações.
6. Envolvam os trechos condicionais ou repetidos com alt, opt ou loop, com guardas entre colchetes.
7. Confiram: toda mensagem recebida é um método da classe do receptor? A ordem respeita as regras de negócio?

4.6 O nível de detalhe certo

Detalhado demais (cada getter, cada clique, cada consulta ao banco de dados), ninguém consegue ler. Genérico demais (“:App → :Sistema: fazTudo()”), o diagrama não mostra a colaboração entre os objetos. O ponto certo é aquele em que aparecem as chamadas que mostram as REGRAS e as INTEGRAÇÕES do cenário. Se o diagrama ficar muito comprido, separem em cenários ou usem ref. De 3 a 6 participantes costumam bastar.

5. Exemplo resolvido: Biblioteca “Ler Mais” — emprestar um livro

Ator: Bibliotecário. Participantes: :SistemaBiblioteca, :Associado, :Livro e :Emprestimo. O sistema pergunta ao :Associado se ele pode pegar mais um livro (a regra dos 3 livros) e ao :Livro se ele está disponível. Se as duas respostas forem sim, pega o prazo (14 dias, ou 30 para professores) e cria o :Emprestimo; senão, devolve a recusa ao bibliotecário.

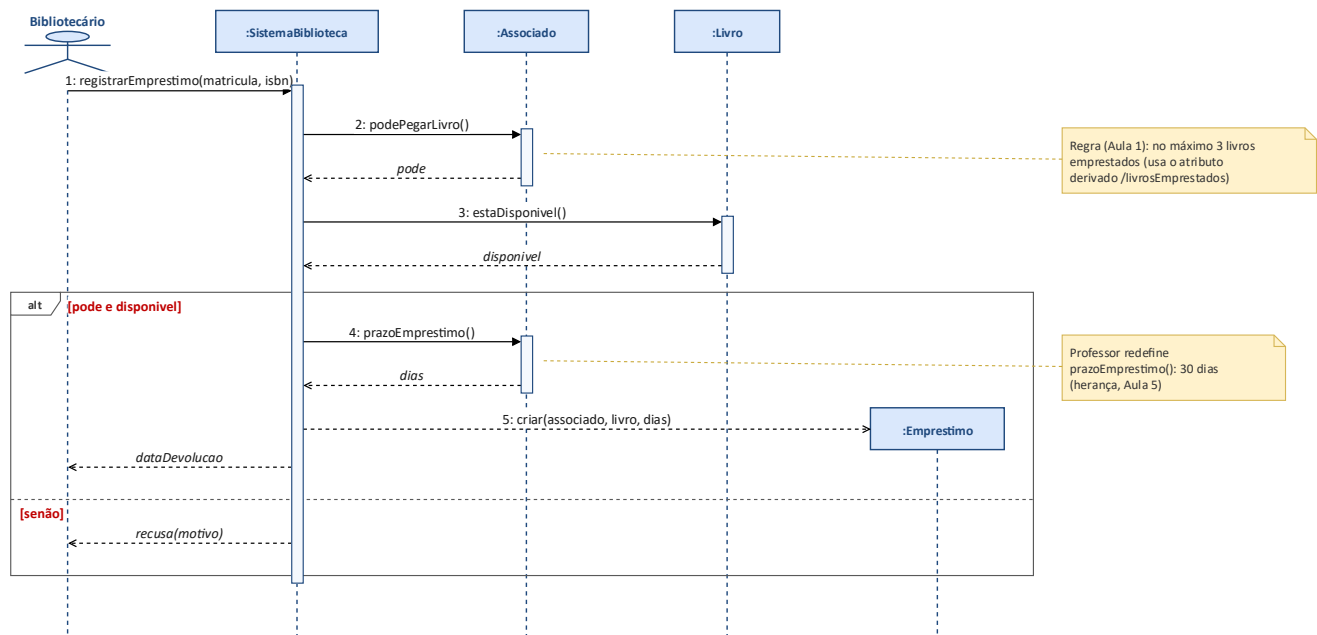


Figura 6 — Biblioteca “Ler Mais”: emprestar um livro.

Leitura: “2: podePegarLivro()” usa o atributo derivado /livrosEmprestados — é a regra dos 3 livros, a mesma do “0..3” da Aula 6. O alt tem duas guardas: [pode e disponivel] e [senão]. “4: prazoEmprestimo()” devolve 14 dias, ou 30 quando o associado é professor, porque a subclasse Professor redefine esse método (herança, Aula 5). Em “5: criar(...)” o :Emprestimo nasce — por isso a caixa dele está mais abaixo que as outras.

6. Atividade guiada: pontos de fidelidade do TechBurger

Vamos construir juntos, ao vivo, no draw.io, um pedaço do pedido pelo app da Aula 7: a ação “Credita os pontos de fidelidade”. Lá ela era uma caixa só; aqui ela vira uma conversa entre objetos, seguindo a Regra 2 da Aula 1 (1 ponto a cada R\$ 10,00; ao atingir 100 pontos, combo grátis na próxima compra e pontuação zerada). Primeiro conheça a tela do draw.io (6.1); depois siga o passo a passo (6.2).

6.1 Onde fica cada coisa no draw.io

Antes de desenhar, vale conhecer a tela. Os números da figura abaixo mostram os seis lugares que você vai usar hoje:

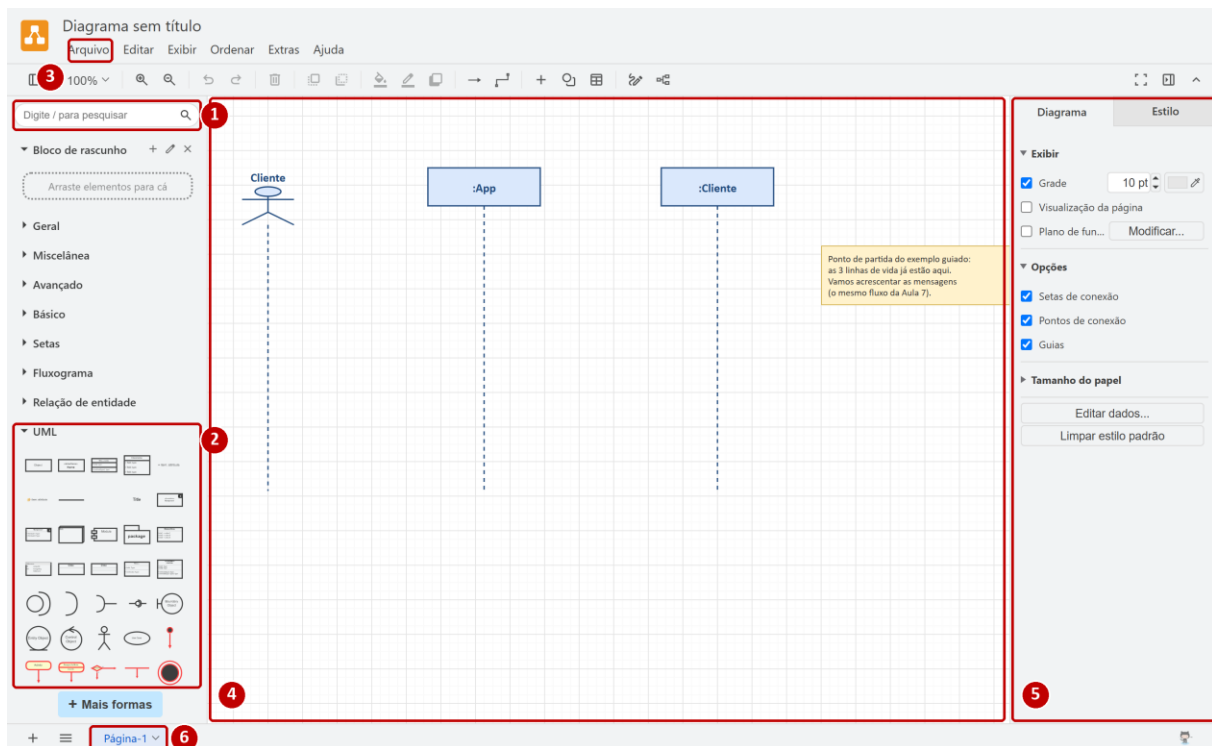
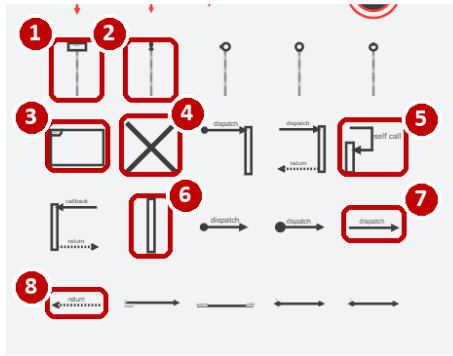


Figura 7 — A tela do draw.io com o ponto de partida da atividade guiada aberto.

1. Pesquisa de formas: digite o nome da forma (em inglês) e aperte Enter. Ex.: “lifeline”.
2. Paleta UML: clique no nome “UML” e role a barra até o FIM da lista — as formas de sequência ficam depois das de atividades.
3. Menu Arquivo: “Abrir” carrega os arquivos de ponto de partida; “Salvar como” guarda o .drawio; “Exportar como > PNG” gera uma imagem para entregar.
4. Área de desenho: nos pontos de partida, as linhas de vida já estão prontas — falta desenhar as mensagens.
5. Painel da direita: com uma seta selecionada, mostra as abas Estilo, Texto e Organizar. É nele que se troca o tipo da seta.
6. Páginas: as abas da parte de baixo. O botão “+” cria outra página no mesmo arquivo.

Na paleta UML, ao parar o mouse sobre um ícone, o draw.io mostra o nome dele em inglês. A Figura 8 mostra os oito ícones que você vai usar:

Paleta UML, rolada até o fim (formas de sequência)



- 1 Participante**
ícone "Lifeline" (caixa :Objeto + linha tracejada)
- 2 Ator**
ícone "Actor Lifeline" (bonequinho + linha)
- 3 Fragmento alt / opt / loop**
ícone "Frame" (moldura com etiqueta)
- 4 Destruição**
ícone "Destruction" (o X)
- 5 Auto-mensagem**
ícone "Self Call"
- 6 Ativação**
ícone "Activation Bar" (retângulo fino)
- 7 Mensagem síncrona**
ícone "Message" (ponta cheia)
- 8 Retorno**
ícone "Return" (tracejada, ponta aberta)

Figura 8 — As formas do diagrama de sequência no fim da paleta UML.

Veja agora cada peça num diagrama pronto (o alt do login da seção 3.1):

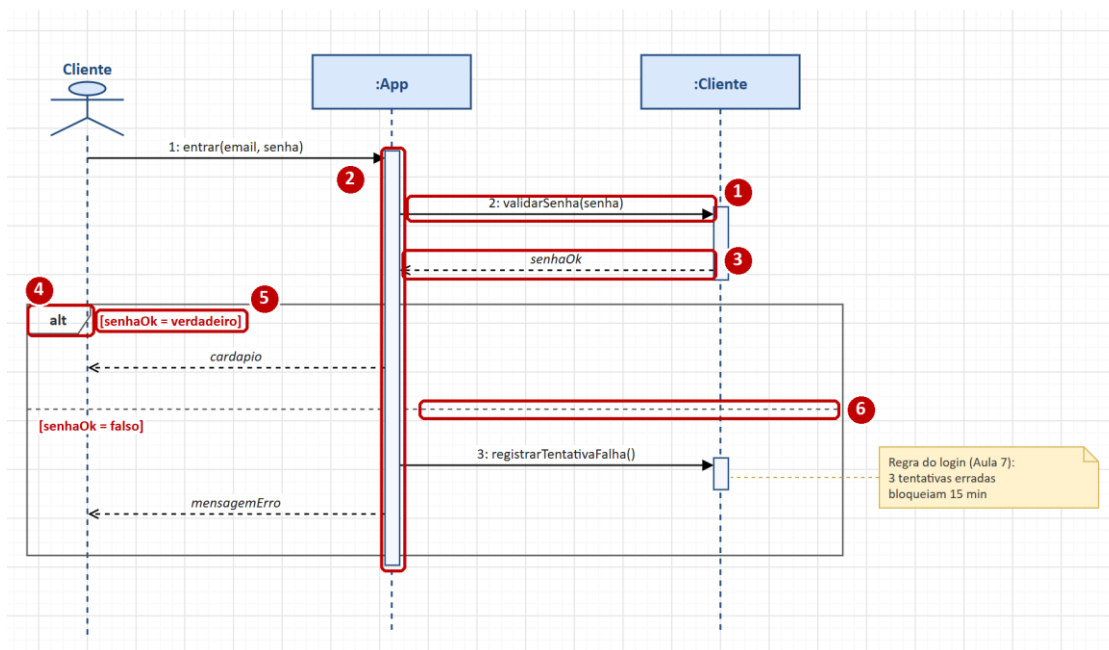


Figura 9 — As peças no diagrama: mensagem (1), ativação (2), retorno (3), moldura do fragmento (4), guarda (5) e divisória do alt (6).

1. Mensagem síncrona: ícone "Message". Arraste para a área de desenho e prenda cada ponta numa linha de vida (ou na ativação): a linha de vida fica destacada quando a ponta está presa.
2. Ativação: ícone "Activation Bar". Solte sobre a linha de vida e estique pelas alças até cobrir o tempo em que o participante trabalha.
3. Retorno: ícone "Return". Vai de quem responde para quem chamou, logo abaixo da chamada.
4. Fragmento: ícone "Frame". Estique em volta das mensagens e dê duplo clique no canto para escrever alt, opt ou loop.

5. Guarda: forma “Text” da paleta Geral, com a condição entre colchetes, ao lado da etiqueta.

6. Divisória do alt: a linha tracejada da paleta Geral, atravessando a moldura de lado a lado.

Mensagem assíncrona e criação de objeto não têm ícone próprio: você desenha uma “Message” (ou um “Return”) e ajusta a seta no painel Estilo, como mostra a Figura 10.



Figura 10 — Com a seta selecionada (1), o painel Estilo muda o padrão da linha (2) e a ponta final (3).

Tabela-resumo — onde fica cada elemento:

Elemento	Onde fica no draw.io	Dica
Participante (objeto)	UML › “Lifeline”	Duplo clique na caixa para escrever “:NomeDaClasse”.
Ator	UML › “Actor Lifeline”	Fica sempre à esquerda; escreva o nome acima do bonequinho.
Ativação	UML › “Activation Bar”	Solte sobre a linha de vida e estique pelas alças.
Mensagem síncrona	UML › “Message”	Ponta cheia. Rótulo: “1: metodo(parametros)”.
Retorno	UML › “Return”	Tracejado, ponta aberta, logo abaixo da chamada.
Mensagem assíncrona	“Message” + Estilo	Troque o “Fim da linha” para a ponta aberta.
Auto-mensagem	UML › “Self Call”	Sai e volta para a mesma linha de vida.
Criação	“Return” até a caixa do novo objeto	Rótulo criar(...); desça a caixa do objeto até a altura da seta.
Destruição	UML › “Destruction”	O X no fim da linha de vida.
alt / opt / loop	UML › “Frame”	Duplo clique no canto para escrever o tipo; guarda com Geral › “Text”.

Atalhos e cuidados:

- Abrir o ponto de partida: Arquivo › Abrir. As linhas de vida já estão lá.
- Número e nome vão juntos no rótulo: duplo clique na seta e digite “1: entrar(email, senha)”.
- Ponta solta? Arraste até a linha de vida ficar destacada — só então a seta está presa.
- Precisa de espaço? Estique as linhas de vida para baixo pelas alças azuis.
- A moldura do fragmento está escondendo as setas? Selecione-a e use Ordenar › Para trás (Ctrl+Shift+B).
- Ctrl+D duplica (ótimo para retornos parecidos); Ctrl+Z desfaz. Salve com um nome que identifique o trabalho.

6.2 Passo a passo da atividade guiada

A história. Na Aula 7, o diagrama do pedido pelo app tinha a ação “Credita os pontos de fidelidade”. Agora vamos abrir essa ação e mostrar QUEM faz O QUÊ, e em que ordem. Usaremos um caso concreto: a Maria tem 95 pontos e acabou de pagar uma compra de R\$ 62,00. O pagamento já foi aprovado e fica de fora deste diagrama (ele é assunto da Atividade 1).

Quem é quem: o ator Cliente (o boneco, à esquerda) é a PESSOA Maria, com o celular na mão; :App é o aplicativo do TechBurger, que recebe o toque e repassa o trabalho; :Cliente é o objeto com os DADOS da Maria dentro do sistema (pontosFidelidade = 95). Da Aula 5, a classe Cliente tem - pontosFidelidade: Inteiro, + acumularPontos(valor: Real) e + resgatarCombo(): Booleano. Quem calcula os pontos é o próprio :Cliente, porque o atributo é privado (-): só ele pode mexer nos pontos (encapsulamento).

Primeiro no papel: antes de abrir o draw.io, escreva a lista de mensagens na ordem em que acontecem:

Nº	De → Para	Mensagem	Por quê
1	Cliente → :App	finalizarCompra()	A Maria toca no botão e espera a resposta (síncrona).
2	:App → :Cliente	acumularPontos(valorPago)	O app pede aos dados da Maria para somar os pontos (valorPago = R\$ 62,00).
3	:Cliente → :Cliente	calcularPontos(valorPago)	Auto-mensagem: $62 \div 10 = 6$ pontos → $95 + 6 = 101$.
4	:Cliente → :Cliente	resgatarCombo()	Só se [saldo $\geq$ 100 pontos] → fica dentro de um opt.
—	:Cliente → :App	saldoAtualizado	Retorno (tracejado) da mensagem 2.
—	:App → Cliente	saldo	Retorno (tracejado) da mensagem 1: a Maria vê o saldo na tela.

Agora no draw.io. Cada passo abaixo corresponde a um slide da aula, em que o que entra naquele passo aparece em laranja.

Passo 0 — Abrir o ponto de partida. Arquivo › Abrir › “Aula 8 - Exemplo Guiado (ponto de partida).drawio”. Confira as 3 linhas de vida: o boneco Cliente, :App e :Cliente. Não é preciso criar participantes.

Passo 1 — A Maria toca em “Finalizar compra”. Arraste da paleta UML o ícone “Message”; prenda uma ponta no tracejado do boneco e a outra no tracejado do :App (a linha de vida fica destacada quando a ponta está presa). Dê duplo clique na seta e escreva “1: finalizarCompra()”. Depois solte um “Activation Bar” sobre o :App, começando na ponta da seta, e estique bem para baixo: o :App trabalha até devolver o saldo.

Passo 2 — O app pede ao :Cliente para acumular. Outra “Message”, um pouco abaixo, saindo da ativação do :App até o tracejado do :Cliente, com o rótulo “2: acumularPontos(valorPago)”. Coloque um “Activation Bar” no :Cliente a partir da ponta da seta. Confira: acumularPontos existe na classe Cliente da Aula 5? Sim.

Passo 3 — O :Cliente calcula os próprios pontos. Use o ícone “Self Call” sobre a ativação do :Cliente: a seta sai e volta para ele mesmo. Rótulo: “3: calcularPontos(valorPago)”. Ao lado, uma nota amarela (Geral › Note) com a regra “1 ponto a cada R\$ 10,00”. No caso da Maria: 6 pontos, saldo 101. Atenção: calcularPontos NÃO estava na classe Cliente da Aula 5 — o diagrama de sequência revelou um método que faltava. Anote na classe: - calcularPontos(valor: Real): Inteiro (privado, porque só o próprio Cliente o usa).

Passo 4 — Só se chegar a 100: o combo. Arraste um “Frame” para baixo da mensagem 3, dê duplo clique no canto e escreva “opt”. Ao lado da etiqueta, uma forma “Text” (paleta Geral) com a guarda [saldo $\geq$ 100 pontos]. Dentro da moldura, outro “Self Call” com “4: resgatarCombo()” e uma nota “100 pontos = combo grátis e saldo zerado”. Se a moldura cobrir as setas, selecione-a e use Ctrl+Shift+B. Por que opt e não alt? Porque, se o saldo não chegar a 100, simplesmente não acontece nada — não existe “senão”.

Passo 5 — As respostas voltam. Um “Return” do :Cliente para o :App, abaixo da moldura, com “saldoAtualizado”; outro “Return” do :App para o boneco, com “saldo”. Retornos são tracejados e voltam para

quem chamou. Ajuste as ativações para terminarem onde sai o retorno. A Maria vê saldo 0: os pontos zeraram quando ela ganhou o combo.

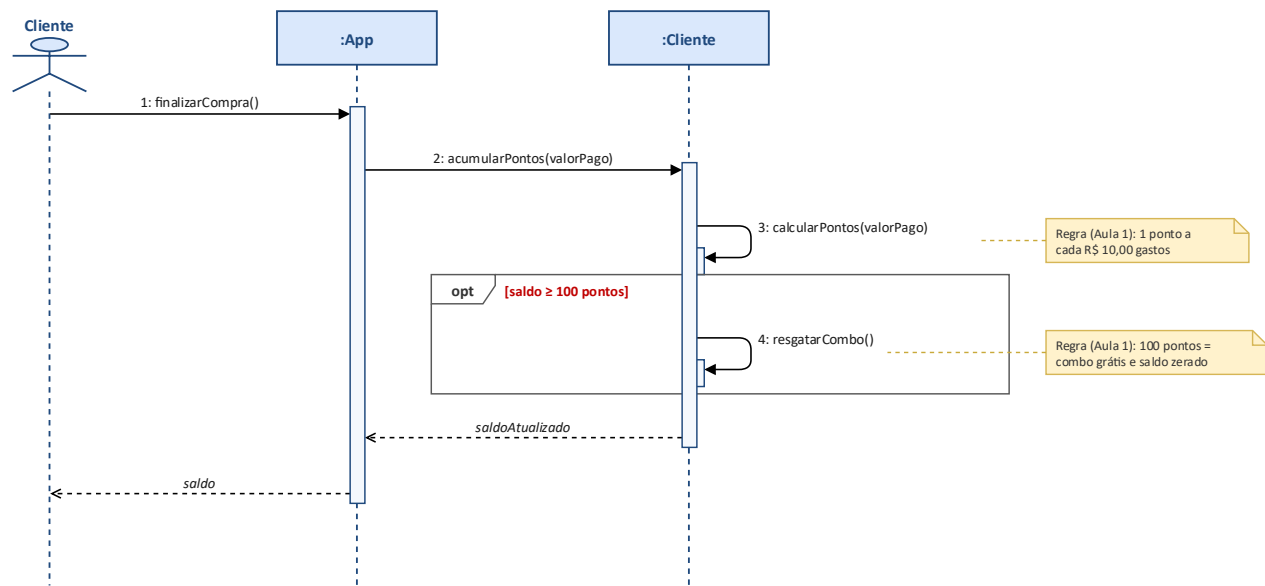


Figura 11 — Resultado da atividade guiada: acumular pontos de fidelidade.

Conferindo com dois clientes: o mesmo diagrama serve para clientes diferentes. Maria: 95 pontos + R\$ 62,00 → 6 pontos → 101; a guarda é verdadeira, o opt acontece, ela ganha o combo e o saldo vai a 0. João: 40 pontos + R\$ 35,00 → 3 pontos → 43; a guarda é falsa, o opt é pulado: acontecem as mensagens 1, 2, 3 e os retornos (saldo 43), mas não a 4.

Checklist da atividade guiada:

- O ator Cliente está à esquerda e inicia a conversa com “1: finalizarCompra()”?
- Todas as setas têm número e nome, e as pontas estão presas nas linhas de vida?
- acumularPontos e resgatarCombo são métodos da classe Cliente (Aula 5)? calcularPontos foi anotado como método novo?
- Os retornos são tracejados, logo abaixo, voltando para quem chamou?
- O opt envolve só o resgatarCombo() e tem a guarda [saldo ≥ 100 pontos]?
- A ação “Credita os pontos de fidelidade” da Aula 7 virou as mensagens 2 a 4?

7. Atividade prática 1 — Pedido pelo app com PIX (TechBurger, em grupo)

Em grupos de 3 a 4 pessoas, desenhem (draw.io ou papel) o diagrama de sequência do cenário “pedido para entrega, pago com PIX”. Partam do arquivo “Aula 8 - Atividade 1 TechBurger (ponto de partida).drawio”, que já traz os 6 participantes: Cliente (ator), :App, :Pedido, :Pagamento, :SistemaPagamento e :Cozinha. Tempo: 30 minutos. Ao final, um representante apresenta à turma.

O cenário, como contado pela gerente Fernanda:

1. O cliente adiciona itens ao carrinho, um de cada vez; o app repassa cada item ao pedido.

2. Depois, o cliente finaliza, informando entrega e endereço; o app pede ao pedido para se validar, e o pedido calcula o próprio valor total. (Neste cenário, o valor passa de R\$ 15,00, os itens estão em estoque e o endereço está a até 5 km — os desvios já estão no diagrama de atividades da Aula 7.)
3. O cliente escolhe pagar com PIX: nesse momento o app cria o objeto Pagamento, com o valor e a forma de pagamento.
4. O app pede ao pagamento para aplicar o desconto de 5%.
5. O app cobra o valor com desconto no Sistema de Pagamento e espera o resultado.
6. Se aprovado: o app confirma o pagamento e o pedido; se o pedido tiver mais de 10 itens, aguarda a confirmação do gerente; depois envia o pedido para a cozinha — sem esperar o preparo — e avisa o cliente.
7. Se recusado: o app cancela o pedido e avisa o cliente.

Atenção: nem todo método de que vocês vão precisar já está nas classes da Aula 5 — `validar()` e `cancelar()`, por exemplo, não estão no Pedido. Podem usá-los: anotem como métodos novos, como fizemos com `calcularPontos()` na atividade guiada.

Exigido: os 6 participantes; mensagens numeradas, com nomes de métodos; retornos tracejados; um loop para os itens; um alt para aprovado/recusado; a criação do `:Pagamento` no meio do diagrama; a mensagem para a cozinha como assíncrona. Bônus: o opt dos 10 itens.

Antes de desenhar, preencham a tabela de planejamento — uma linha por mensagem (as três primeiras já estão preenchidas como exemplo de formato):

Nº	De	Para	Mensagem	Tipo
1	Cliente	:App	adicionarItem(produto, qtd)	síncrona (dentro do loop)
2	:App	:Pedido	adicionarItem(produto, qtd)	síncrona (dentro do loop)
—	:Pedido	:App	itemAdicionado	retorno

No draw.io, para esta atividade (tabela da seção 6.1):

- Arquivo › Abrir: “Aula 8 - Atividade 1 TechBurger (ponto de partida).drawio”.
- loop e alt: “Frame” (paleta UML) — duplo clique no canto para escrever loop / alt; guardas com a forma “Text” da paleta Geral.
- cobrar(...) e o seu “resultado”: “Message” e, logo abaixo, “Return”.
- O envio à :Cozinha é assíncrono: “Message” com o “Fim da linha” trocado para a ponta aberta (painel Estilo).
- Criação do :Pagamento: um “Return” até a caixa do :Pagamento — e desçam a caixa até a altura do criar(...).
- A linha que separa [aprovado] de [recusado] no alt: a linha tracejada da paleta Geral.

Checklist da atividade 1

- Os 6 participantes, com o ator à esquerda.
- Mensagens numeradas, com nomes de métodos (verbo + parênteses).
- Retornos tracejados logo abaixo das chamadas síncronas.
- loop e alt com guardas entre colchetes (e, de bônus, o opt).
- :Pagamento criado no meio do diagrama.
- Ordem que respeita as regras: validar → pagar → só então a cozinha.

Dicas para o grupo

- Primeiro escrevam a lista numerada de mensagens no papel: quem → quem: o quê.
- Dividam as tarefas: um lê o cenário, outro desenha, outro confere o checklist.
- Em cada seta perguntem: “quem envia espera a resposta?” — isso decide entre síncrona e assíncrona.
- Leiam o diagrama em voz alta, de cima para baixo, antes de apresentar.

8. Exercício: encontre o erro

Em duplas, analisem o diagrama abaixo. Ele tem 6 erros propositais. Para cada um: (1) marquem onde está, (2) expliquem POR QUE é um erro — qual regra desta apostila foi violada — e (3) escrevam como ficaria a correção. Tentem primeiro sem consultar o resto da apostila. Respostas no Gabarito.

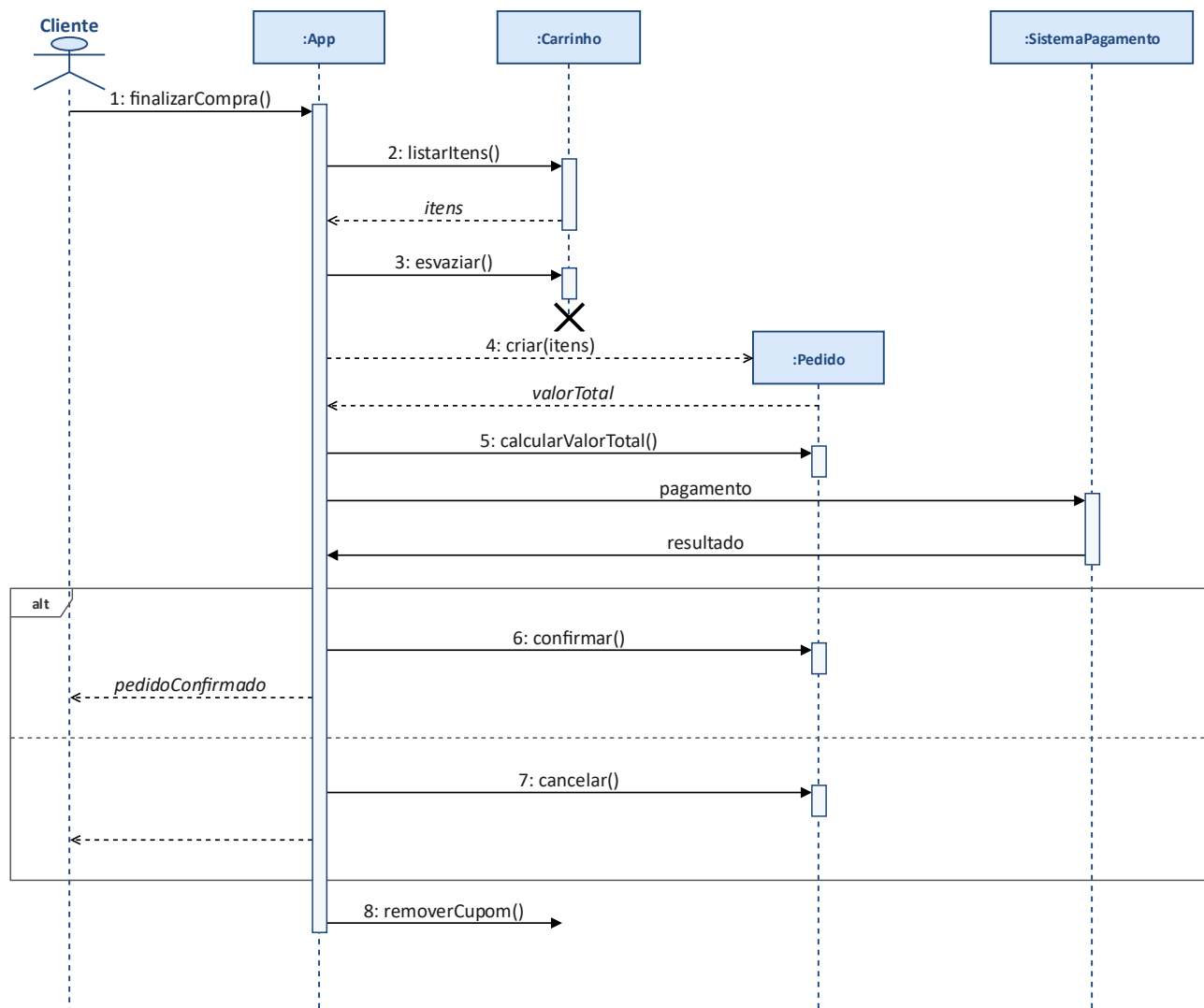


Figura 12 — Diagrama de sequência com 6 erros propositais.

Dica: percorram a tabela de elementos da seção 2 e perguntem, seta por seta: o tipo de seta está certo? A mensagem é um método (verbo)? Tem nome? A resposta vem depois da chamada? O objeto já existe (ou ainda existe) quando recebe a mensagem? Os fragmentos têm guardas?

9. Atividade prática 2 — Revisão final das Aulas 1 a 8 (CineVila, em duplas)

Na prática profissional, tudo o que vimos até aqui é feito em sequência, sobre o mesmo sistema: primeiro entendemos o negócio (regras), depois dizemos o que o sistema deve fazer (requisitos e critérios de aceite) e só então desenhamos os modelos UML (casos de uso, classes, atividades e sequência). Nesta atividade vocês vão fazer esse caminho completo em um domínio novo. Tempo: 40 minutos, em duplas, já contando a correção coletiva. Na próxima aula (15/10) faremos a revisão integradora completa com outro cenário, a EcoBike.

9.1 O cenário CineVila

O CineVila é um cinema de bairro que quer vender ingressos pelo aplicativo. O cliente escolhe um filme, uma sessão (data, horário e sala) e os assentos. Cada compra tem no máximo 6 ingressos. Os assentos escolhidos ficam reservados por 10 minutos, esperando o pagamento; se o pagamento não acontecer nesse prazo, eles

são liberados. Estudantes e idosos pagam meia-entrada (50%) e apresentam o documento na entrada. O pagamento é feito no app, por cartão ou PIX, num sistema de pagamento externo. Com o pagamento aprovado, o app emite os ingressos com QR code. O cliente pode cancelar a compra até 2 horas antes da sessão. O mapa de assentos precisa abrir em até 2 segundos, e o app deve funcionar em Android e iOS.

9.2 Relembrando rapidamente cada aula

Aula	Conceito	Formato que vamos usar hoje
1	Regra de negócio	Condição (o “se”) / Ação (o “então”) / Exceção (quando não vale)
2	Requisito funcional e não funcional	RF: “O sistema deve [verbo]...”. RNF: qualidade mensurável (tempo, plataforma, segurança)
3	Critério de aceite; estrutural x comportamental	Dado [contexto] / Quando [ação] / Então [resultado esperado]
4	Casos de uso	Atores fora da fronteira; casos de uso com verbo; «include» = sempre; «extend» = só às vezes
5	Classes	Nome / atributos (- nome: tipo) / métodos (+ nome())
6	Cardinalidade	1, 0..1, 0..* (*), 1..* e faixas como 0..3 nas duas pontas; herança sem números
7	Atividades	Nó inicial, ações, decisões com guardas, junção, raias, nó final
8	Sequência	Participantes, mensagens numeradas, retornos tracejados, alt/opt/loop

9.3 Tarefas

- **Parte A (Aula 1):** identifiquem 3 regras de negócio e escrevam cada uma como Condição / Ação / Exceção.
- **Parte B (Aula 2):** escrevam 3 requisitos funcionais e 2 requisitos não funcionais.
- **Parte C (Aula 3):** escrevam 1 critério de aceite (Dado / Quando / Então) para a regra dos 10 minutos, e classifiquem os diagramas de casos de uso, classes, atividades e sequência em estruturais ou comportamentais.
- **Parte D (Aula 4):** listem os atores e pelo menos 4 casos de uso, com 1 «include» ou «extend».
- **Parte E (Aulas 5 e 6):** definam pelo menos 4 classes com 2 atributos e 1 método cada, e os relacionamentos entre elas COM cardinalidade.
- **Parte F (Aula 7):** desenhem o diagrama de atividades “Comprar ingresso”, com raias e pelo menos 2 decisões com guardas.
- **Parte G (Aula 8):** desenhem o diagrama de sequência “Comprar ingresso” (pagamento aprovado ou recusado), com pelo menos 4 participantes e 1 alt.

Sugestão de tempo: 6 minutos para as partes A a C, 9 minutos para D e E, 9 minutos para F e 10 minutos para G; os minutos finais são da correção coletiva. Usem as tabelas abaixo para as partes A e E:

Regra (resumo)	Condição	Ação	Exceção

Classe A	Card. A	Relacionamento	Card. B	Classe B

No draw.io, para esta atividade (tabela da seção 6.1):

- Arquivo › Novo. Usem duas páginas no mesmo arquivo (botão “+” embaixo): uma para a parte F, outra para a G.
- Parte F (atividades, Aula 7): “Start”, “Activity”, “Condition” e “End” da paleta UML; raias com “Vertical Container” da paleta Geral.
- Parte G (sequência): “Actor Lifeline” para o Cliente e “Lifeline” para cada objeto (:Compra, :Sessao...).
- Parte G: alt [aprovado] / [recusado] com “Frame” + “Text” + linha tracejada.

Checklist da atividade 2

- Cada regra tem condição e ação claras (e exceção, quando existir)?
- Os RF dizem O QUE o sistema faz; os RNF dizem COMO ele se comporta (tempo, plataforma)?
- A regra dos 6 ingressos aparece como cardinalidade (E) e como decisão (F)?
- As mensagens do diagrama de sequência (G) são métodos das classes da parte E?
- A regra dos 10 minutos aparece nas partes A, C, F e G?

10. Erros comuns ao modelar sequência

- Retorno desenhado com seta contínua, ou chamada síncrona desenhada com linha tracejada.
- Mensagem com substantivo (“pagamento”) ou sem nome.
- Ordem errada: resposta acima da chamada, ou cobrar o cliente antes de validar o pedido.
- Mensagem para um método que não existe na classe de quem recebe.
- Objeto recebendo mensagens antes de ser criado ou depois do X.
- Fragmento sem guarda, ou alt usado onde bastava um opt.
- Participante que não existe no diagrama de classes (ou ator desenhado como objeto).

11. Boas práticas

- Um diagrama por cenário, com um nome claro (“Pedido com PIX aprovado”).
- Ator à esquerda; participantes na ordem em que entram na conversa.
- Mensagens numeradas e com os nomes exatos dos métodos das classes.
- Retornos só quando o resultado importa — e sempre tracejados.
- Revisar junto com o diagrama de classes e com o de atividades: um corrige o outro.

Glossário

Termo	Significado
Diagrama de sequência	Diagrama comportamental da UML que mostra a troca de mensagens entre participantes, ao longo do tempo.
Participante	Objeto (“:Classe”) ou ator que participa da conversa.
Linha de vida	Linha tracejada vertical: a existência do participante ao longo do tempo.
Ativação	Retângulo fino sobre a linha de vida: o período em que o participante está

	trabalhando.
Mensagem síncrona	Seta contínua de ponta cheia: chamada em que quem envia espera a resposta.
Mensagem assíncrona	Seta contínua de ponta aberta: quem envia não espera.
Mensagem de retorno	Seta tracejada: devolve o resultado para quem chamou.
Auto-mensagem	O objeto chama um método dele mesmo.
Criação / destruição	O objeto nasce na altura da seta de criação / deixa de existir no X.
Fragmento combinado	Moldura alt, opt, loop, par ou ref, com condição de guarda.
Cenário	Um caminho específico de um caso de uso (ex.: pedido com PIX aprovado).

Perguntas de revisão

1. Qual a diferença entre o diagrama de atividades e o de sequência? Quando usar cada um?
2. O que representa a linha de vida? E a ativação?
3. Qual a diferença entre mensagem síncrona, assíncrona e de retorno? Dê um exemplo de cada.
4. Por que a mensagem para a cozinha é assíncrona e a cobrança do PIX é síncrona?
5. Como se representa a criação de um objeto? E a destruição?
6. Qual a diferença entre os fragmentos alt, opt e loop?
7. Qual a relação entre as mensagens do diagrama de sequência e os métodos das classes?
8. Como uma regra de negócio da Aula 1 aparece no diagrama de sequência? Dê um exemplo.

Quiz relâmpago para casa

Cinco perguntas para conferir o que ficou. Respostas no Gabarito.

1. Para que lado corre o tempo no diagrama de sequência?
2. Qual seta tem ponta cheia: a síncrona ou a assíncrona?
3. Como é desenhado o retorno?
4. Qual fragmento usar para “se o cliente tiver cupom, aplica o desconto”?
5. Cite uma mensagem de hoje que é um método do diagrama de classes da Aula 5.